Secure Password Storage System

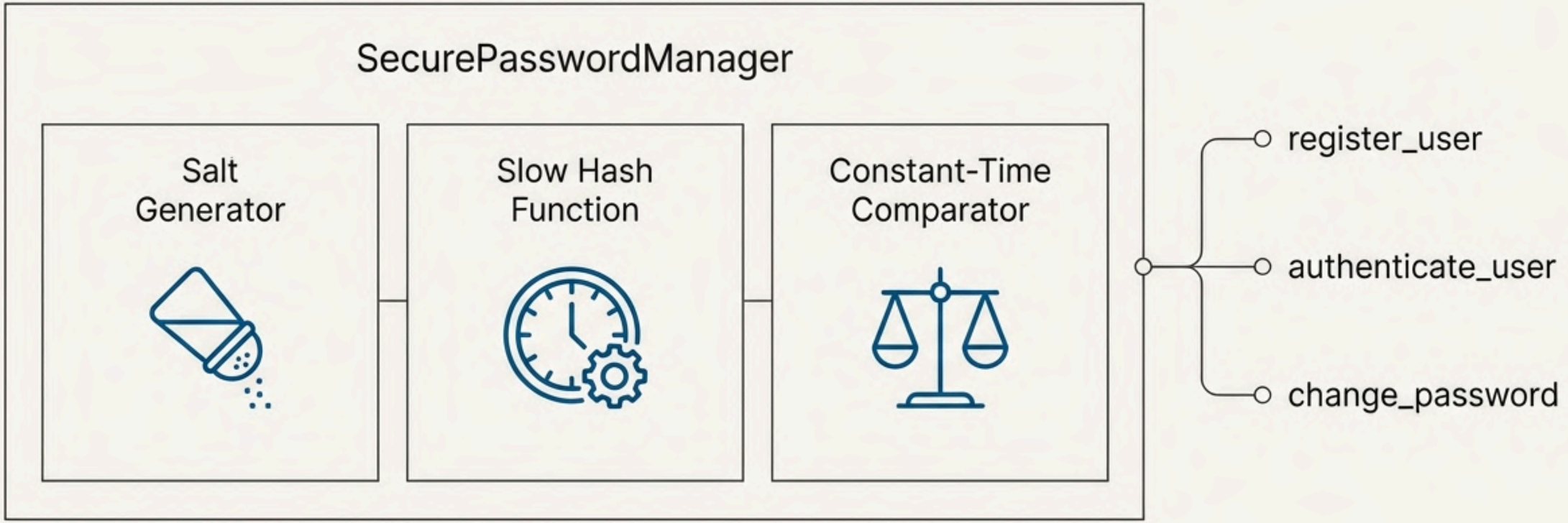
Implementation from Scratch



Final Project - Software Security

Programming-Based Project (Option B)

We build a fortress, not a façade.



A multi-layered system designed from first principles to defend against modern threats.

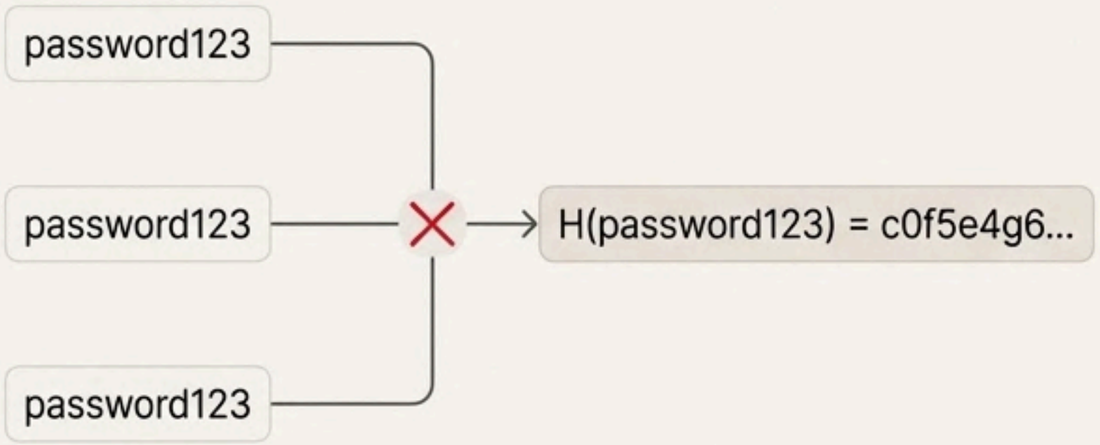
NotebookLM

Layer 1: The Salted Moat.

Defeating pre-computed attacks before they begin.

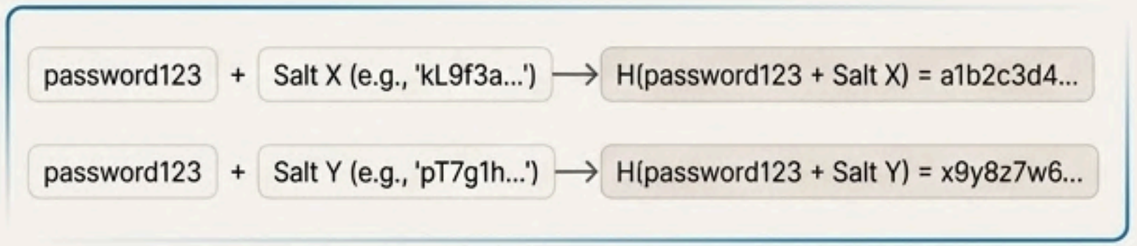
Attack: Rainbow Tables

Pre-computed hash tables that make common password hashes instantly crackable.



Defense: Unique Salting

A unique, random salt is added to every password. This ensures that even identical passwords have unique hashes, rendering rainbow tables useless.



Evidence

```
def generate_salt(self, length: int = 32) -> bytes:
    return os.urandom(32)
```

Key Stat

32 bytes of cryptographically secure random data provide 2^{256} possible salts. Uniqueness is practically guaranteed.

NotebookLM

Layer 2: The Labyrinthine Walls

Making brute-force attacks impossibly expensive

The goal is to make a single password guess so slow for an attacker that a large-scale attack becomes infeasible.



The Slow Key Derivation Function

- We use an iterative hashing process, similar in principle to PBKDF2.
- Each password undergoes **100,000 rounds** of SHA-256 hashing by default.
- Crucially, each round depends on the last, preventing attackers from using parallel processing (like GPUs) to speed up the cracking process.

NotebookLM

The Anatomy of the Slow Hash.

Initial hash combines password and unique salt.

The core loop: 99,999 iterations make it computationally expensive.

```
def slow_hash(self, password: str, salt: bytes, iterations: int) -> bytes:
    password_bytes = password.encode('utf-8')

    current_hash = hashlib.sha256(password_bytes + salt).digest()

    for i in range(iterations - 1):
        current_hash = hashlib.sha256(
            current_hash + password_bytes + salt + i.to_bytes(4, 'big')
        ).digest()

    return current_hash[:self.hash_length]
```

HMAC-like construction re-injects the password, salt, and a counter in each round, ensuring every iteration is unique and sequential.

NotebookLM

The 0.3 Second Delay: A Feature, Not a Bug.

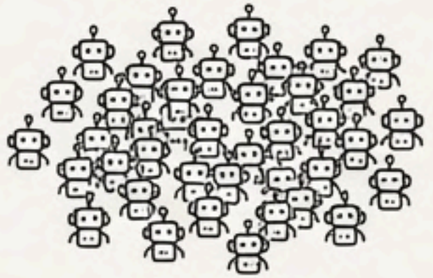
Legitimate Login



~0.3 - 0.5 seconds

An acceptable, barely noticeable delay for a real user.

Brute-Force Attack (1 Billion Guesses)



~3,000 - 5,000 years

(on a single CPU)

The 100,000x slowdown makes brute force economically and practically infeasible.

NotebookLM

Layer 3: The Silent Guard

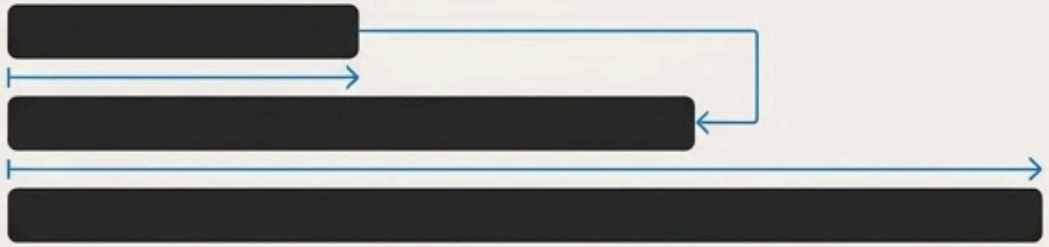
Defending against an attack you can't even see

Attack: Timing Side-Channel

Standard string comparisons (`==`) can stop early when a difference is found. By precisely measuring response times, an attacker can guess a hash character by character.



==



Defense: Constant-Time Comparison

We use a comparison function that always takes the same amount of time to execute, regardless of whether the hashes match or where the first difference occurs. It leaks zero timing information.



`_constant_time_compare`



NotebookLM

A Fortress Built for Purpose.

	Attack	Defense
	Rainbow Tables	Unique Salt Per Password
	Brute Force / Dictionary Attacks	Slow Hashing (100,000 Iterations)
	Timing Attacks	Constant-Time Comparison
	User Enumeration	Dummy Hash for Non-Existent Users

Defense in depth: Every layer is designed to nullify a specific class of attack.

NotebookLM

Validation: The System Under Test

Registration & Authentication

✓ Register 'alice' (took **0.352s**)
✓ Authenticate 'alice' (took **0.350s**)

Handling Failure

✗ Correctly rejected 'alice' with wrong password (took **0.349s**)
✗ Correctly rejected 'nonexistent' user (took **0.351s**)

Password Update

✓ Password changed for 'alice'
✗ Old password rejected
✓ New password accepted

> **Key Takeaway**
>
> Note the consistent timing across success and failure cases—proof that the timing attack defense is working.

NotebookLM

Measuring Against Industry Standards.

Feature	This Implementation	Industry Standard (bcrypt)
Algorithm	Custom PBKDF2-like (SHA-256)	Blowfish-based
Salt Size	32 bytes (256-bit)	16 bytes (128-bit)
Iterations	Configurable (100k default)	Configurable (cost factor)
Core Principle	Educational / Proof of Concept	Battle-Tested / Widely Deployed

Our implementation correctly mirrors the core principles of established standards like PBKDF2 and bcrypt: **configurable computational cost**, **integrated salting**, and **timing attack resistance**.



NotebookLM

Reinforcing the Fortress for the Future



Memory-Hard Functions (e.g., scrypt, Argon2)
To resist brute-force attacks from specialized hardware like GPUs and ASICs by requiring significant memory.



Server-Side Pepper
A secret key stored on the server, added to every hash. It provides another layer of protection if the database is compromised.



Adaptive Cost
Automatically increasing the iteration count over time to keep pace with Moore's Law, ensuring the fortress remains secure against future hardware.



Account Lockout & Rate Limiting
Practical defenses to slow down online attacks against individual accounts.

NotebookLM

Anatomy of a Secure Password System

THE REGISTRATION FLOW: STORING A NEW PASSWORD



1. Generate a Unique "Salt"

A cryptographically secure 32-byte random value is created for each new password.

2. Apply a Slow Hash Function

The password and salt are hashed together 100,000 times to make it computationally expensive.

3. Store the Results Securely

Only the unique salt and the final computed hash are stored, never the plaintext password.

THE AUTHENTICATION FLOW: VERIFYING A LOGIN ATTEMPT



1. Retrieve Stored Salt & Hash

The system looks up the user's unique salt and previously stored hash.

2. Combine and Hash

The submitted password is combined with the stored salt and hashed 100,000 times again.

3. Compare Hashes Securely

The new hash is compared to the stored hash using a constant-time method.

HOW THESE STEPS DEFEND AGAINST ATTACKS

Each security layer is designed to defeat a specific attack vector. This creates a defense-in-depth strategy for password protection.



NotebookLM


Thank
You!